



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ

ΕΡΓΑΣΤΗΡΙΟ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

Ψηφιακά Συστήματα VLSI

2η Εργαστηριακή Άσκηση

Σχεδίαση Μονάδων Υλικού με την Τεχνική Pipelining

Ομάδα 6

Μέλος 1:

Ονοματεπώνυμο: Γιώργος Μπάρκας

Αριθμός Μητρώου: el22139

Μέλος 2:

Ονοματεπώνυμο: Δημήτρης Βλασακάκης

Αριθμός Μητρώου: el22818

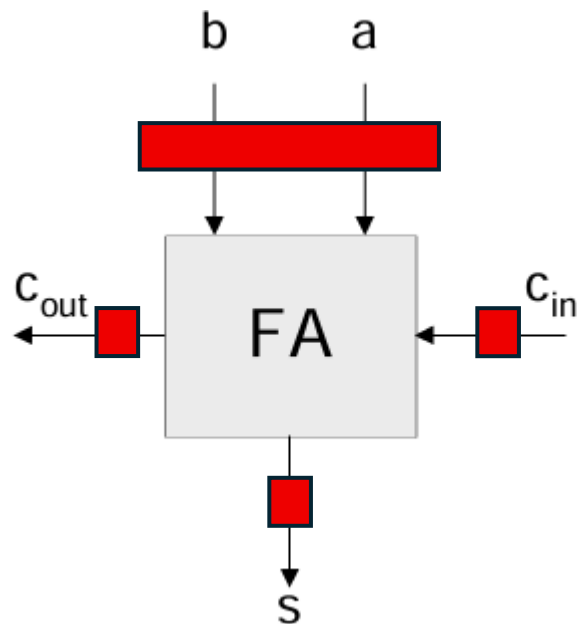
Σχεδίαση Μονάδων Υλικού με την Τεχνική Pipelining

Στην αναφορά παρουσιάζεται η σχεδίαση σύγχρονων υπολογιστικών κυκλωμάτων κάνοντας χρήση της τεχνικής pipeline. Τα κυκλώματα αυτά αποτελούνται από διαφορετικά υποσυστήματα που μπορούν να επεξεργάζονται παράλληλα διαφορετικό υποσύνολο δεδομένων. Η λειτουργία αυτή επιτυγχάνεται με την εισαγωγή μονάδων καθυστέρησης (D flip-flop) ανάμεσα στα υποσυστήματα.

Θέμα 1: Υλοποιήστε έναν Σύγχρονο Πλήρη Αθροιστή (Full Adder - FA) με περιγραφή συμπεριφοράς (Behavioral).

Η σχεδίαση του σύγχρονου πλήρη αθροιστή μας, θα βασιστεί στο παρακάτω διάγραμμα:

D flip-flop: 



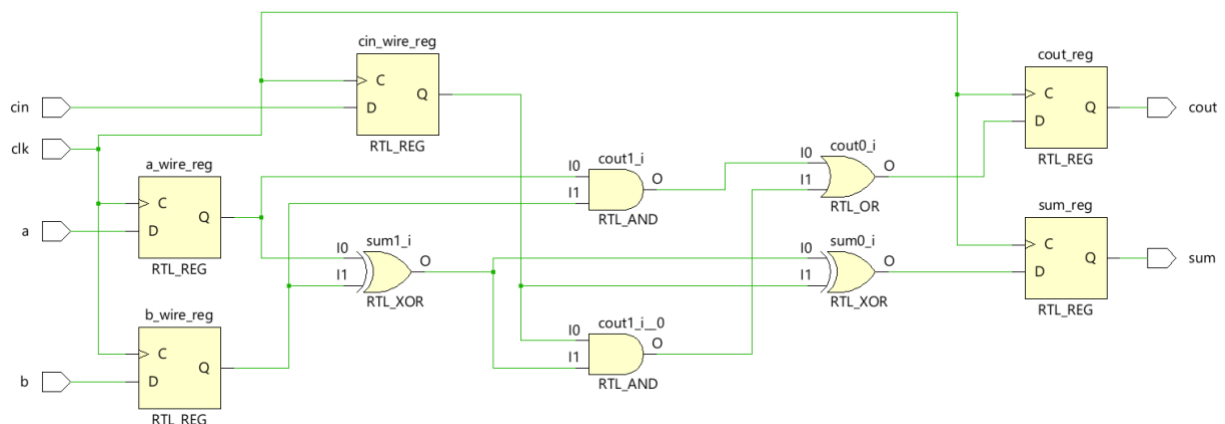
Η υλοποίηση μας είναι η παρακάτω:

```

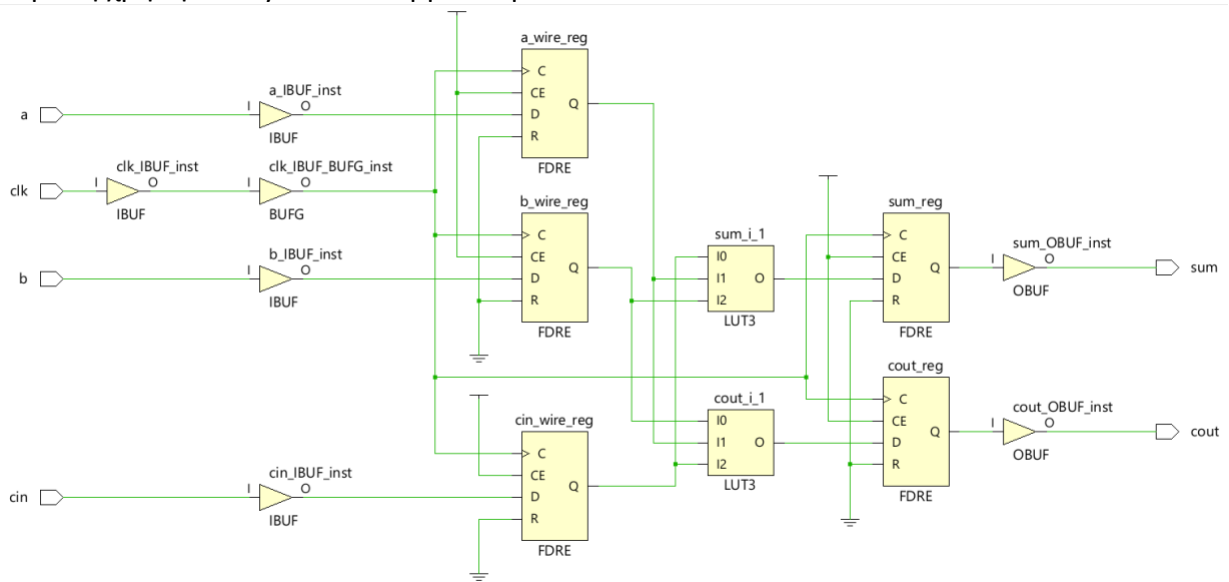
22  library IEEE;
23  use IEEE.STD_LOGIC_1164.ALL;
24
25  entity full_sync_adder is
26      Port ( a : in STD_LOGIC;
27            b : in STD_LOGIC;
28            cin : in STD_LOGIC;
29            clk : in STD_LOGIC;
30            sum : out STD_LOGIC;
31            cout : out STD_LOGIC);
32  end full_sync_adder;
33
34  architecture Behavioral of full_sync_adder is
35      signal a_wire,b_wire,cin_wire: std_logic;
36      begin
37
38      process(clk)
39      begin
40          --process for full sync adder
41          if clk'event and clk='1' then
42              a_wire <= a;
43              b_wire <= b;
44              cin_wire <= cin;
45          --process for sum
46          if ( (a_wire xor b_wire xor cin_wire) = '1' ) then
47              sum <= '1';
48          else
49              sum <= '0';
50          end if;
51          --process for cout
52          if ( ( (a_wire and b_wire) or ( cin_wire and (a_wire xor b_wire) ) ) = '1' ) then
53              cout <= '1';
54          else
55              cout <= '0';
56          end if;
57          end if;
58      end process;
59  end Behavioral;
60

```

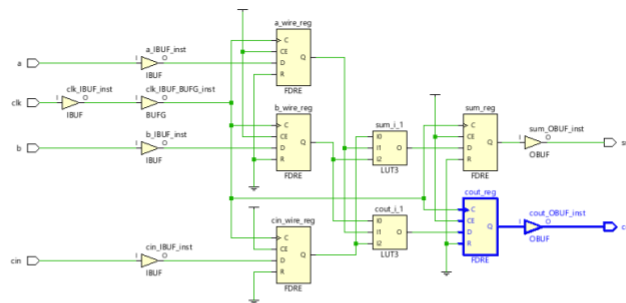
Έτσι, το RTL schematic, δηλαδή το δομικό διάγραμμα θα είναι:



Και με τη χρήση του synthesis λαμβάνουμε:



Εδώ μπορούμε να βρούμε και το critical path μας όπου:



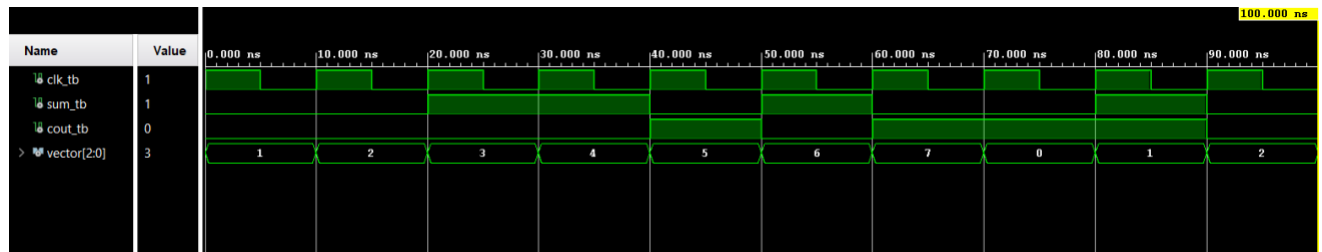
Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	∞	2	2	1	cout_reg/C	cout	4.076	3.276	0.800	∞				0.000
Path 2	∞	2	2	1	sum_reg/C	sum	4.076	3.276	0.800	∞				0.000
Path 3	∞	1	2	1	a	a_wire_reg/D	1.782	0.982	0.800	∞	input port clock			0.000
Path 4	∞	1	2	1	b	b_wire_reg/D	1.782	0.982	0.800	∞	input port clock			0.000
Path 5	∞	1	2	1	cin	cin_wire_reg/D	1.782	0.982	0.800	∞	input port clock			0.000
Path 6	∞	2	2	2	cin_wire_reg/C	sum_reg/D	1.529	0.777	0.752	∞				0.000
Path 7	∞	2	2	2	b_wire_reg/C	cout_reg/D	1.503	0.751	0.752	∞				0.000

Με χρονική καθυστέρηση 4.076ns.

Τέλος, θέλουμε να δημιουργήσουμε ένα σχετικό testbench, ώστε να γίνει ο έλεγχος της ορθής λειτουργίας του κυκλώματος μας:

```
26 entity full_sync_adder_tb is
27 end full_sync_adder_tb;
28
29 architecture arch_full_sync_adder_tb of full_sync_adder_tb is
30 component full_sync_adder is
31     port( a,b,cin,clk: in std_logic;
32           sum,cout: out std_logic);
33 end component;
34
35 signal clk_tb,sum_tb,cout_tb: std_logic;
36 signal vector: std_logic_vector(2 downto 0) := "000";
37 begin
38 uut:full_sync_adder
39     port map(a => vector(0), b => vector(1), cin => vector(2),
40             clk => clk_tb,
41             sum => sum_tb,cout => cout_tb);
42
43 clk:process
44 begin
45     clk_tb <= '1';
46     wait for 5ns;
47
48     clk_tb <= '0';
49     wait for 5ns;
50 end process;
51
52 tb:process(clk_tb)
53 begin
54     if clk_tb'event and clk_tb='1' then
55         if vector /=7 then
56             vector <= vector + 1;
57         else
58             vector <= "000";
59         end if;
60     end if;
61 end process;
62 end arch_full_sync_adder_tb;
63
```

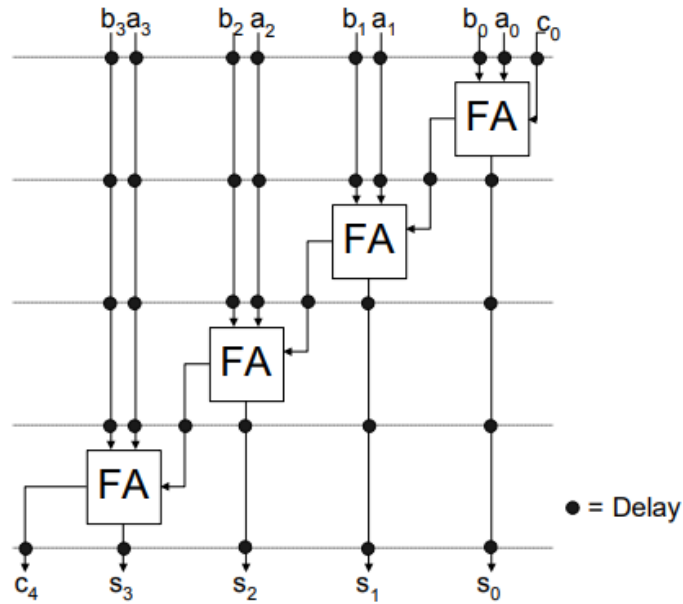
Τρέχοντας την προσομοίωση, λαμβάνουμε:



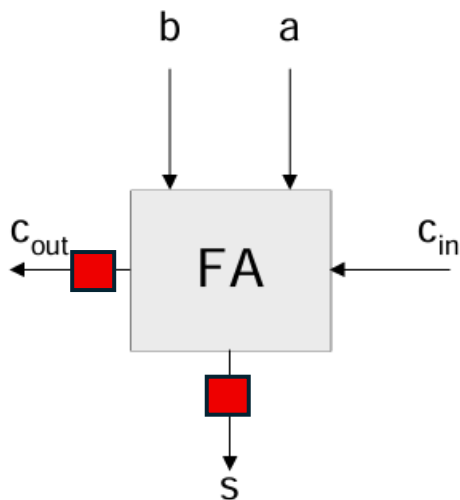
Παρατηρούμε ότι στον πρώτο θετικό παλμό φορτώνεται η είσοδος στον full adder και μετά από ένα κύκλο παράγεται η έξοδος.

2) Υλοποιήστε έναν σύγχρονο Αθροιστή διάδοσης κρατουμένου των 4 bits με χρήση της τεχνικής Pipeline.

Παρακάτω φαίνεται το διάγραμμα του σύγχρονο αθροιστή διάδοσης κρατουμένου των 4 bits το οποίο αποτελείται από τέσσερα pipelines:

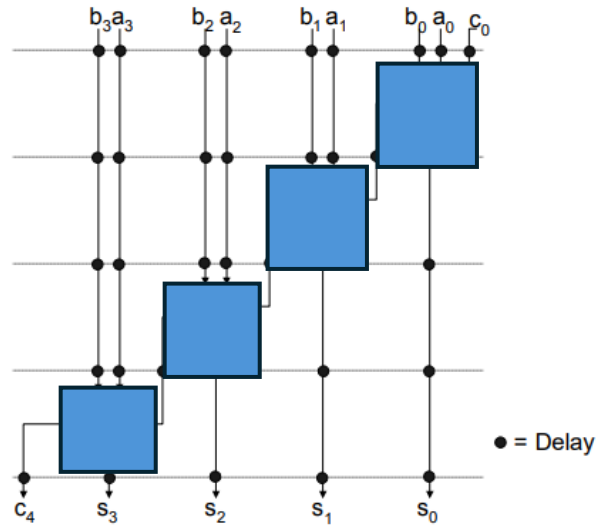


Για να περιγράψουμε αυτό το διάγραμμα σε structural αρχιτεκτονική θα πρέπει να το χωρίσουμε σε components. Βλέπουμε ότι αποτελείται από 4 *full_adder*, άρα θα χρειαστούμε 4 πανομοιότυπα components, τα οποία έχουν την εξής μορφή:



Άρα η τελική σχεδίαση μας θα βασιστεί στο παρακάτω διάγραμμα:

Component: 



Η υλοποίηση μας είναι η παρακάτω:

```

23 use IEEE.STD_LOGIC_1164.ALL;
24
25 entity full_sync_adder is
26     Port ( a : in STD_LOGIC;
27           b : in STD_LOGIC;
28           cin : in STD_LOGIC;
29           clk : in STD_LOGIC;
30           sum : out STD_LOGIC;
31           cout : out STD_LOGIC);
32 end full_sync_adder;
33
34 architecture Behavioral of full_sync_adder is
35 begin
36
37 process(clk)
38 begin
39     if clk'event and clk='1' then
40         if ( (a xor b xor cin) = '1' ) then
41             sum <= '1';
42         else
43             sum <= '0';
44         end if;
45         if ( ( (a and b) or ( cin and (a xor b) ) ) = '1' ) then
46             cout <= '1';
47         else
48             cout <= '0';
49         end if;
50     end if;
51 end process;
52 end Behavioral;

```

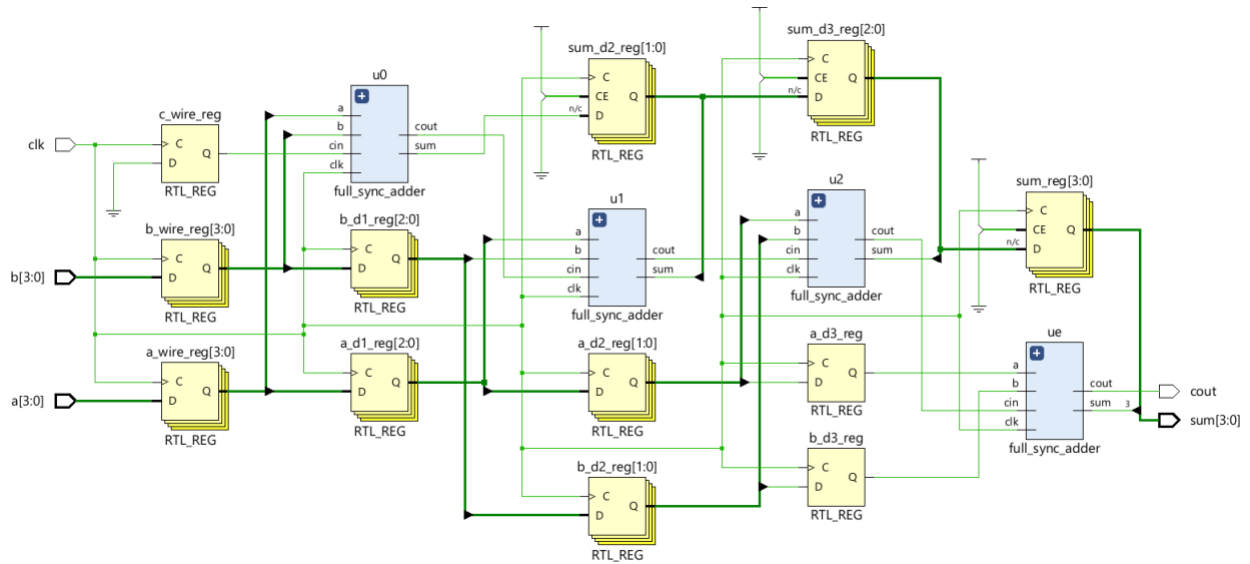

Έτσι βασιζόμενοι στην λογική που περιγράψαμε παραπάνω και στον *full_adder*, υλοποιήσαμε τον σύγχρονο αθροιστή μας ως εξής:

```

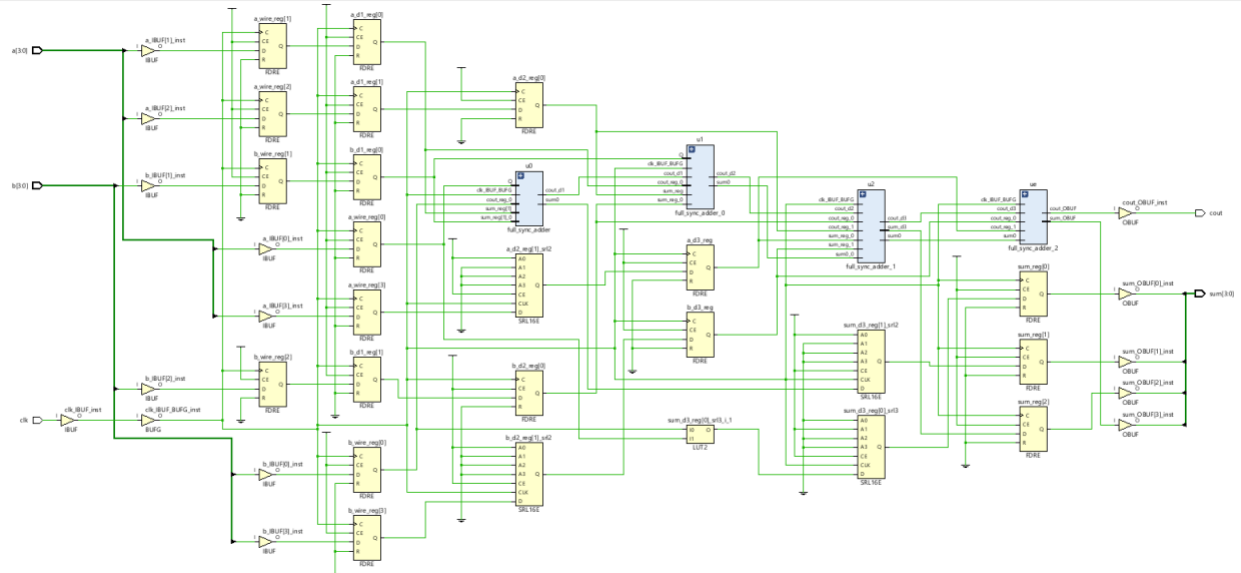
22 library IEEE;
23 use IEEE.STD_LOGIC_1164.ALL;
24
25 entity parallel_sync_adder is
26     Port ( a : in STD_LOGIC_VECTOR (3 downto 0);
27           b : in STD_LOGIC_VECTOR (3 downto 0);
28           clk : in STD_LOGIC;
29           cout : out STD_LOGIC;
30           sum : out STD_LOGIC_VECTOR (3 downto 0));
31 end parallel_sync_adder;
32
33 architecture structural of parallel_sync_adder is
34     component full_sync_adder is
35         Port ( a : in STD_LOGIC;
36               b : in STD_LOGIC;
37               clk : in STD_LOGIC;
38               cin : in std_logic;
39               sum : out STD_LOGIC;
40               cout : out STD_LOGIC);
41     end component;
42     signal a_wire,b_wire,sum_wire,cout_wire,sum_d4: std_logic_vector(3 downto 0);
43     signal a_d1,b_d1,sum_d3: std_logic_vector(2 downto 0);
44     signal a_d2,b_d2,sum_d2: std_logic_vector(1 downto 0);
45     signal cout_d1,cout_d2,cout_d3,a_d3,b_d3,sum_d1,c_wire:std_logic;
46
47     begin
48         u0 : full_sync_adder port map (a => a_wire(0),b => b_wire(0),cin => c_wire ,sum => sum_d1,cout => cout_d1, clk => clk);
49         u1 : full_sync_adder port map (a => a_d1(0),b => b_d1(0),cin => cout_d1,sum => sum_d2(1),cout => cout_d2, clk => clk);
50         u2 : full_sync_adder port map (a => a_d2(0),b => b_d2(0),cin => cout_d2,sum => sum_d3(2),cout => cout_d3, clk => clk);
51         ue : full_sync_adder port map (a => a_d3,b => b_d3,cin => cout_d3,sum => sum(3),cout => cout, clk => clk);
52
53     H0:process(clk)
54     begin
55         if clk'event and clk='1' then
56             a_wire <= a;
57             b_wire <= b;
58             c_wire <= '0';
59
60             a_d1 <= a_wire(3 downto 1);
61             b_d1 <= b_wire(3 downto 1);
62
63             a_d2 <= a_d1(2 downto 1);
64             b_d2 <= b_d1(2 downto 1);
65             sum_d2(0) <= sum_d1;
66
67             a_d3 <= a_d2(1);
68             b_d3 <= b_d2(1);
69             sum_d3(0) <= sum_d2(0);
70             sum_d3(1) <= sum_d2(1);
71
72             sum(0) <= sum_d3(0);
73             sum(1) <= sum_d3(1);
74             sum(2) <= sum_d3(2);
75         end if;
76     end process;
77 end structural;

```

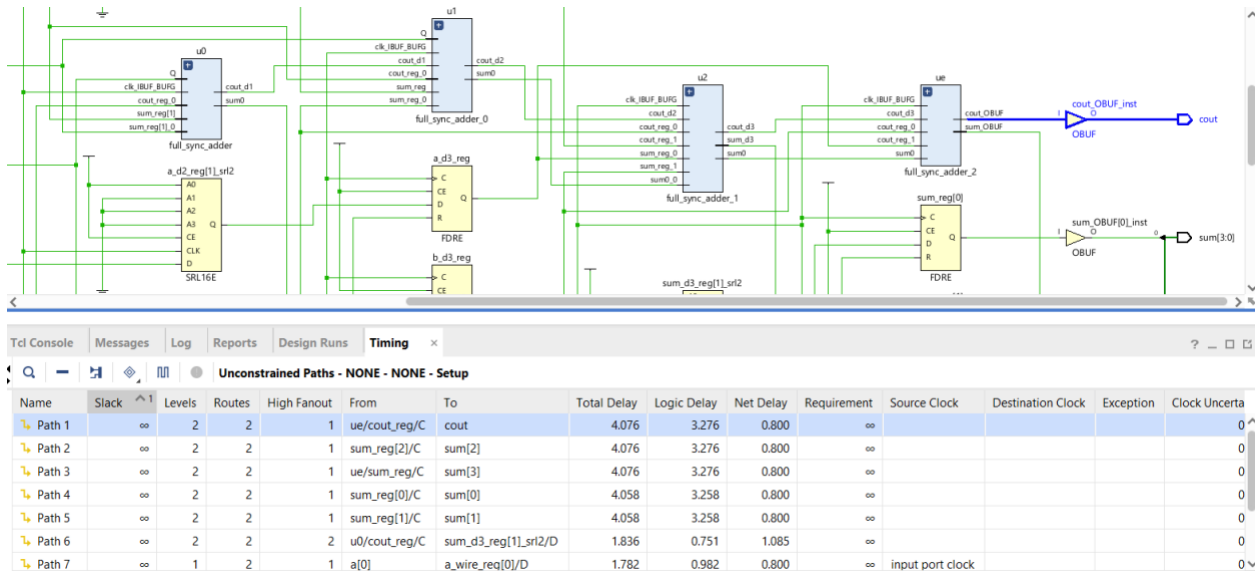
Έτσι, το RTL schematic, δηλαδή το δομικό διάγραμμα θα είναι:



Και με τη χρήση του synthesis λαμβάνουμε:



Εδώ μπορούμε να βρούμε και το critical path μας όπου:



Με χρονική καθυστέρηση 4.076ns.

Ενδιαφέρον είναι η σύγκριση του critical path του σύγχρονου αθροιστή με αυτό του ασύγχρονου παράλληλου αθροιστή που υλοποιήθηκε στην εργαστηριακή άσκηση 1.

Critical path ασύγχρονου παράλληλου αθροιστή:

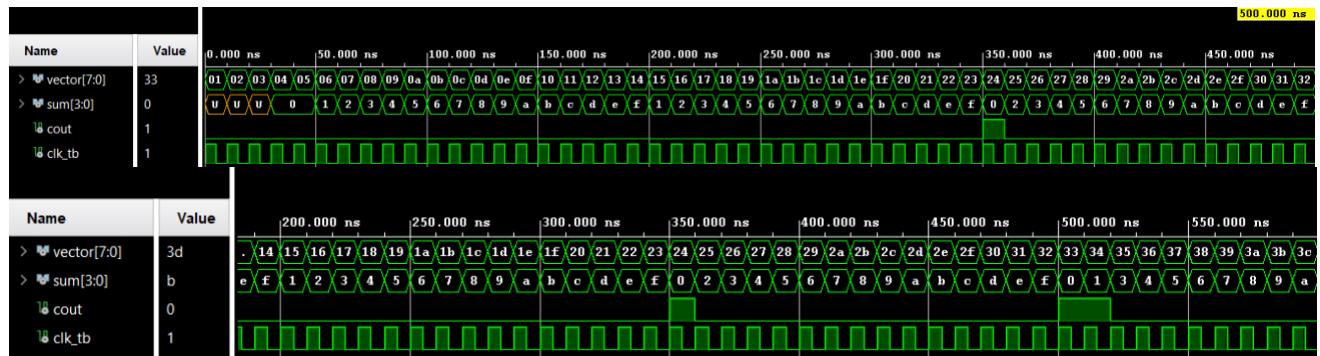
Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	3	4	2	in2	sum	5.255	3.656	1.599
Path 2	∞	3	4	2	in2	sum	5.255	3.656	1.599
Path 3	∞	3	4	2	in2	sum	5.255	3.656	1.599
Path 4	∞	3	4	2	in2	c_out	5.229	3.630	1.599
Path 5	∞	3	4	2	in1	c_out	5.229	3.630	1.599
Path 6	∞	3	4	2	c_in	c_out	5.229	3.630	1.599

Παρατηρείται ότι το critical path του σύγχρονου αθροιστή είναι μικρότερο από αυτό του ασύγχρονου παράλληλου αθροιστή. Η μειωμένη χρονική καθυστέρηση είναι αποτέλεσμα της pipeline σχεδίασης. Συγκεκριμένα με την εισαγωγή μονάδων καθυστέρησης (ανάμεσα στα υποσυστήματα) κάθε υποσύστημα ενεργεί πλέον ανεξάρτητα από τα υπόλοιπα, με αποτέλεσμα να μειώνεται ο χρόνος για τον οποίον η είσοδος πρέπει να παραμένει σταθερή για την παραγωγή ορθού αποτελέσματος.

Τέλος, θέλουμε να δημιουργήσουμε ένα σχετικό testbench, ώστε να γίνει ο έλεγχος της ορθής λειτουργίας του κυκλώματος μας:

```
22 library IEEE;
23 use IEEE.STD_LOGIC_1164.ALL;
24 use IEEE.std_logic_unsigned.all;
25
26 entity parallel_sync_adder_tb is
27 end parallel_sync_adder_tb;
28
29 architecture arch_parallel_sync_adder_tb of parallel_sync_adder_tb is
30 component parallel_sync_adder is
31     Port ( a : in STD_LOGIC_VECTOR (3 downto 0);
32           b : in STD_LOGIC_VECTOR (3 downto 0);
33           clk : in STD_LOGIC;
34           cout : out STD_LOGIC;
35           sum : out STD_LOGIC_VECTOR (3 downto 0));
36 end component;
37
38 signal vector: std_logic_vector (7 downto 0):="00000000";
39 signal sum : std_logic_vector(3 downto 0);
40 signal cout,clk_tb: std_logic;
41 begin
42
43 uut:parallel_sync_adder
44 port map(a => vector(3 downto 0), b => vector(7 downto 4),
45          sum => sum, cout => cout, clk => clk_tb);
46
47 clk:process
48 begin
49     clk_tb <= '1';
50     wait for 5ns;
51
52     clk_tb <= '0';
53     wait for 5ns;
54 end process;
55
56 tb:process(clk_tb)
57 begin
58     if clk_tb'event and clk_tb = '1' then
59         if vector /= 255 then
60             vector <= vector + 1;
61         else
62             vector <= "00000000";
63         end if;
64     end if;
65 end process;
66 end arch_parallel_sync_adder_tb;
```

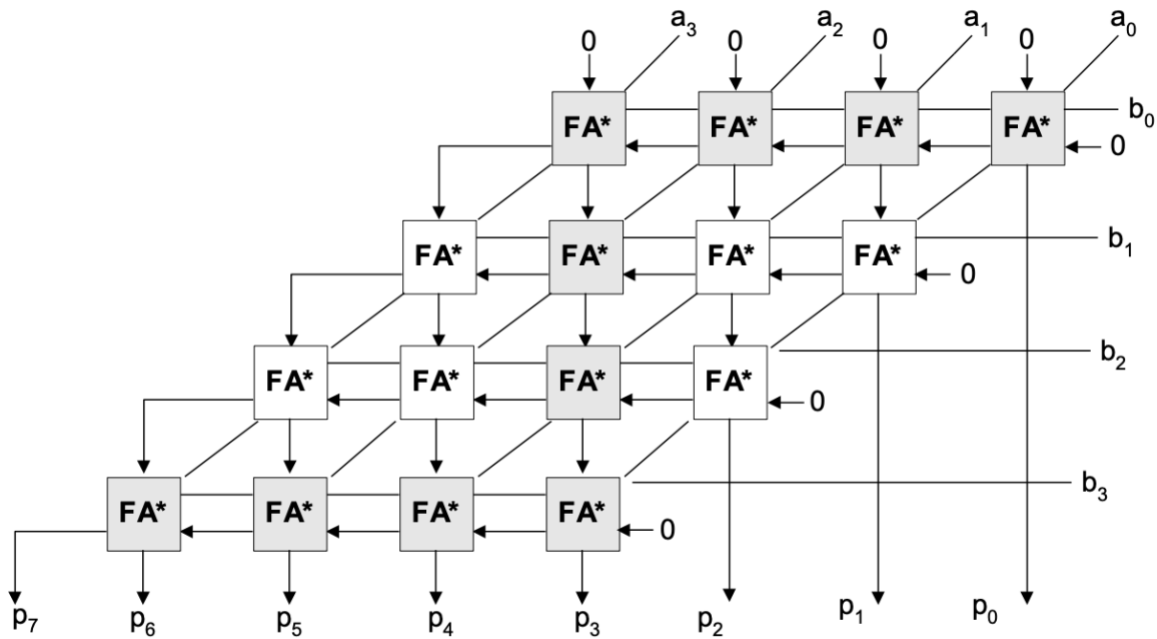
Τρέχοντας την προσομοίωση, λαμβάνουμε:



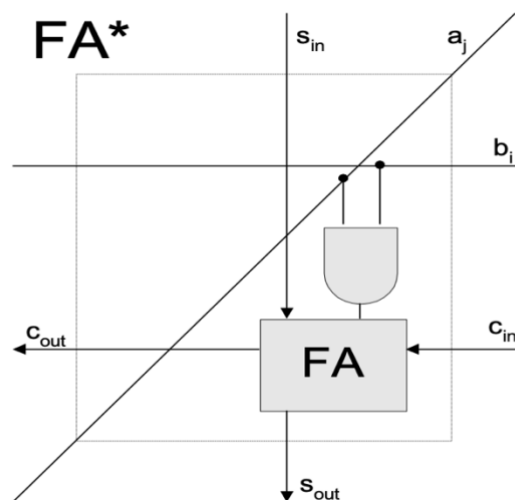
Παρατηρούμε ότι στον πρώτο θετικό παλμό φορτώνεται η είσοδος στον πρώτο full adder και μετά από τέσσερις κύκλοι παράγεται η έξοδος.

Θέμα 3: Συστολικός Πολλαπλασιαστής διάδοσης κρατουμένων των 4 bits

Η σχεδίαση του πολλαπλασιαστή μας, θα βασιστεί στο παρακάτω διάγραμμα:

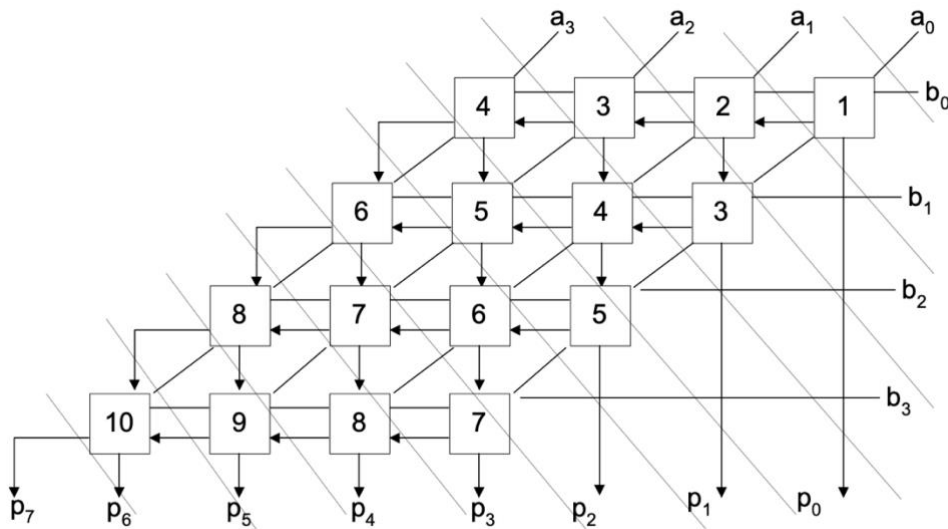


Βλέπουμε ότι αποτελείται από 16 *full_adder_star*, οι οποίοι έχουν την εξής μορφή:

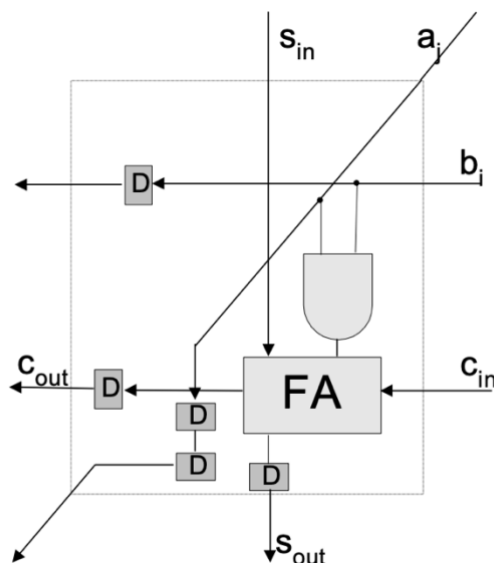


Έτσι, αρχικά θα υλοποιήσουμε τη δομή *full_adder_star* και στη συνέχεια θα χρησιμοποιήσουμε το κύτταρο για την κατασκευή του πολλαπλασιαστή μας.

Ωστόσο, από τη στιγμή που θέλουμε το κύκλωμα μας να είναι σύγχρονο, θα πρέπει τα δεδομένα μας να εισέρχονται και να εξέρχονται τις σωστές χρονικές στιγμές μέσα και έξω από το κύτταρο. Για αυτό παρατηρώντας τη σειρά που πρέπει να γίνουν οι πράξεις, βλέπουμε ότι ακολουθούν τη σειρά που δείχνει η παρακάτω αρίθμηση. Στον πρώτο κύκλο, τα a_0, b_0 θα τραβηχτούν απευθείας χωρίς κάποια καθυστέρηση και θα υλοποιηθεί το κύτταρο 1. Στον δεύτερο κύκλο, το c_{out1} θα είναι έτοιμο και έτσι θα ενεργοποιηθούν τα a_1, b_1 ώστε να υλοποιηθεί το κύτταρο 2, μετά τα κύτταρα 3 και ούτω καθεξής. Συμπερασματικά, το a_i θα πρέπει να καθυστερήσει i κύκλους και το b_j $2 * j$ κύκλους.



Επίσης, εσωτερικά βλέπουμε ότι μετά από κάθε *full_adder_star*, το αποτέλεσμα και το carry αναγκαστικά θα είναι έτοιμα μετά από έναν κύκλο, άρα και το b_j θα πρέπει να καθυστερεί 1 κύκλο, ενώ το a_i 2 κύκλους. Αυτό μπορεί να αναπαρασταθεί ως:



Όπου δεν πρέπει να προστεθεί εξωτερική καθυστέρηση για τα s_{out} , c_{out} αφού προστίθεται απευθείας μέσω του *full adder*.

Η υλοποίηση μας είναι η παρακάτω:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder_star is
    port(
        a : in std_logic;
        b : in std_logic;
        ci : in std_logic;
        si : in std_logic;
        aout : out std_logic;
        bout : out std_logic;
        sout : out std_logic;
        cout : out std_logic;
        clk : in std_logic
    );
end full_adder_star;

architecture full_adder_star_arch of full_adder_star is

    signal res_and : std_logic;
    signal a_d1 : std_logic;

begin
    res_and <= a and b;

    process(clk)
    begin
        if rising_edge(clk) then
            sout <= si xor res_and xor ci;
            cout <= (si and res_and) or (res_and and ci) or (si and ci);

            a_d1 <= a;
            aout <= a_d1;

            bout <= b;
        end if;
    end process;
end full_adder_star_arch;
```

Έτσι βασιζόμενοι στην λογική που περιγράψαμε παραπάνω και στον *full_adder_star*, υλοποιήσαμε τον πολλαπλασιαστή μας ως εξής:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mult is
    port(
        a : in std_logic_vector(3 downto 0);
        b : in std_logic_vector(3 downto 0);
        p : out std_logic_vector(7 downto 0);
        clk : in std_logic
    );
```



```

    );
end mult;

architecture mult_arch of mult is
    component full_adder_star
        port(
            a : in std_logic;
            b : in std_logic;
            ci : in std_logic;
            si : in std_logic;
            aout : out std_logic;
            bout : out std_logic;
            sout : out std_logic;
            cout : out std_logic;
            clk : in std_logic
        );
    end component;

    -- for delaying the input
    signal a1, a2, a2_2, a3, a3_2, a3_3: std_logic;
    signal b1, b2, b3, b1_2, b2_2, b2_3, b2_4, b3_2, b3_3, b3_4, b3_5, b3_6 : std_logic;
    signal cout0, cout1, cout2: std_logic; -- for the carry of the leftmost full adders
    -- _X_temp(Y): X is the bit number and Y the number of pushes
    signal a0_temp, a1_temp, a2_temp, a3_temp: std_logic_vector(3 downto 0);
    signal b0_temp, b1_temp, b2_temp, b3_temp: std_logic_vector(3 downto 0);
    -- _X_temp(Y): X is the line and Y is the column starting from the right
    signal ci0_temp, ci1_temp, ci2_temp, ci3_temp: std_logic_vector(3 downto 0);
    signal s0_temp, s1_temp, s2_temp, s3_temp: std_logic_vector(3 downto 0);

    signal p0_0, p0_1, p0_2, p0_3, p0_4, p0_5, p0_6, p0_7, p0_8: std_logic;
    signal p1_0, p1_1, p1_2, p1_3, p1_4, p1_5, p1_6: std_logic;
    signal p2_0, p2_1, p2_2, p2_3, p2_4: std_logic;
    signal p3_0, p3_1, p3_2: std_logic;
    signal p4_0, p4_1: std_logic;
    signal p5: std_logic;

begin

    fa1 : full_adder_star
        port map (
            a => a(0),
            b => b(0),
            ci => '0',
            si => '0',
            aout => a0_temp(0),
            bout => b0_temp(0),
            sout => s0_temp(0),
            cout => ci0_temp(0),
            clk => clk
        );

    fa2 : full_adder_star
        port map (
            a => a1,
            b => b0_temp(0),
            ci => ci0_temp(0),
            si => '0',
            aout => a1_temp(0),
            bout => b0_temp(1),
            sout => s0_temp(1),
            cout => ci0_temp(1),
            clk => clk
        );

    fa3_up: full_adder_star
        port map (

```

```

        a => a2,
        b => b0_temp(1),
        ci => ci0_temp(1),
        si => '0',
        aout => a2_temp(0),
        bout => b0_temp(2),
        sout => s0_temp(2),
        cout => ci0_temp(2),
        clk => clk
    );
fa3_down: full_adder_star
    port map (
        a => a0_temp(0),
        b => b1,
        ci => '0',
        si => s0_temp(1),
        aout => a0_temp(1),
        bout => b1_temp(0),
        sout => s1_temp(0),
        cout => ci1_temp(0),
        clk => clk
    );
fa4_up: full_adder_star
    port map (
        a => a3,
        b => b0_temp(2),
        ci => ci0_temp(2),
        si => '0',
        aout => a3_temp(0),
        bout => b0_temp(3),
        sout => s0_temp(3),
        cout => ci0_temp(3),
        clk => clk
    );
fa4_down: full_adder_star
    port map (
        a => a1_temp(0),
        b => b1_temp(0),
        ci => ci1_temp(0),
        si => s0_temp(2),
        aout => a1_temp(1),
        bout => b1_temp(1),
        sout => s1_temp(1),
        cout => ci1_temp(1),
        clk => clk
    );
fa5_up: full_adder_star
    port map (
        a => a2_temp(0),
        b => b1_temp(1),
        ci => ci1_temp(1),
        si => s0_temp(3),
        aout => a2_temp(1),
        bout => b1_temp(2),
        sout => s1_temp(2),
        cout => ci1_temp(2),
        clk => clk
    );
fa5_down: full_adder_star
    port map (
        a => a0_temp(1),
        b => b2,
        ci => '0',
        si => s1_temp(1),
        aout => a0_temp(2),

```

```

        bout => b2_temp(0),
        sout => s2_temp(0),
        cout => ci2_temp(0),
        clk => clk
    );
fa6_up: full_adder_star
    port map (
        a => a3_temp(0),
        b => b1_temp(2),
        ci => ci1_temp(2),
        si => cout0,
        aout => a3_temp(1),
        bout => b1_temp(3),
        sout => s1_temp(3),
        cout => ci1_temp(3),
        clk => clk
    );
fa6_down: full_adder_star
    port map (
        a => a1_temp(1),
        b => b2_temp(0),
        ci => ci2_temp(0),
        si => s1_temp(2),
        aout => a1_temp(2),
        bout => b2_temp(1),
        sout => s2_temp(1),
        cout => ci2_temp(1),
        clk => clk
    );
fa7_up: full_adder_star
    port map (
        a => a2_temp(1),
        b => b2_temp(1),
        ci => ci2_temp(1),
        si => s1_temp(3),
        aout => a2_temp(2),
        bout => b2_temp(2),
        sout => s2_temp(2),
        cout => ci2_temp(2),
        clk => clk
    );
fa7_down: full_adder_star
    port map (
        a => a0_temp(2),
        b => b3,
        ci => '0',
        si => s2_temp(1),
        aout => a0_temp(3),
        bout => b3_temp(0),
        sout => s3_temp(0),
        cout => ci3_temp(0),
        clk => clk
    );
fa8_up: full_adder_star
    port map (
        a => a3_temp(1),
        b => b2_temp(2),
        ci => ci2_temp(2),
        si => cout1,
        aout => a3_temp(2),
        bout => b2_temp(3),
        sout => s2_temp(3),
        cout => ci2_temp(3),
        clk => clk
    );

```

```

fa8_down: full_adder_star
  port map (
    a => a1_temp(2),
    b => b3_temp(0),
    ci => ci3_temp(0),
    si => s2_temp(2),
    aout => a1_temp(3),
    bout => b3_temp(1),
    sout => s3_temp(1),
    cout => ci3_temp(1),
    clk => clk
  );
fa9: full_adder_star
  port map (
    a => a2_temp(2),
    b => b3_temp(1),
    ci => ci3_temp(1),
    si => s2_temp(3),
    aout => a2_temp(3),
    bout => b3_temp(2),
    sout => s3_temp(2),
    cout => ci3_temp(2),
    clk => clk
  );
fa10: full_adder_star
  port map (
    a => a3_temp(2),
    b => b3_temp(2),
    ci => ci3_temp(2),
    si => cout2,
    aout => a3_temp(3),
    bout => b3_temp(3),
    sout => s3_temp(3),
    cout => ci3_temp(3),
    clk => clk
  );

process(clk)
begin
  if rising_edge(clk) then

    a1 <= a(1);
    a2 <= a2_2;
    a2_2 <= a(2);
    a3 <= a3_2;
    a3_2 <= a3_3;
    a3_3 <= a(3);

    b1 <= b1_2;
    b1_2 <= b(1);
    b2 <= b2_2;
    b2_2 <= b2_3;
    b2_3 <= b2_4;
    b2_4 <= b(2);
    b3 <= b3_2;
    b3_2 <= b3_3;
    b3_3 <= b3_4;
    b3_4 <= b3_5;
    b3_5 <= b3_6;
    b3_6 <= b(3);

    cout0 <= ci0_temp(3);
    cout1 <= ci1_temp(3);
    cout2 <= ci2_temp(3);

```

```

p0_0 <= p0_1;
p0_1 <= p0_2;
p0_2 <= p0_3;
p0_3 <= p0_4;
p0_4 <= p0_5;
p0_5 <= p0_6;
p0_6 <= p0_7;
p0_7 <= p0_8;
p0_8 <= s0_temp(0);

p1_0 <= p1_1;
p1_1 <= p1_2;
p1_2 <= p1_3;
p1_3 <= p1_4;
p1_4 <= p1_5;
p1_5 <= p1_6;
p1_6 <= s1_temp(0);

p2_0 <= p2_1;
p2_1 <= p2_2;
p2_2 <= p2_3;
p2_3 <= p2_4;
p2_4 <= s2_temp(0);

p3_0 <= p3_1;
p3_1 <= p3_2;
p3_2 <= s3_temp(0);

p4_0 <= p4_1;
p4_1 <= s3_temp(1);

p5 <= s3_temp(2);

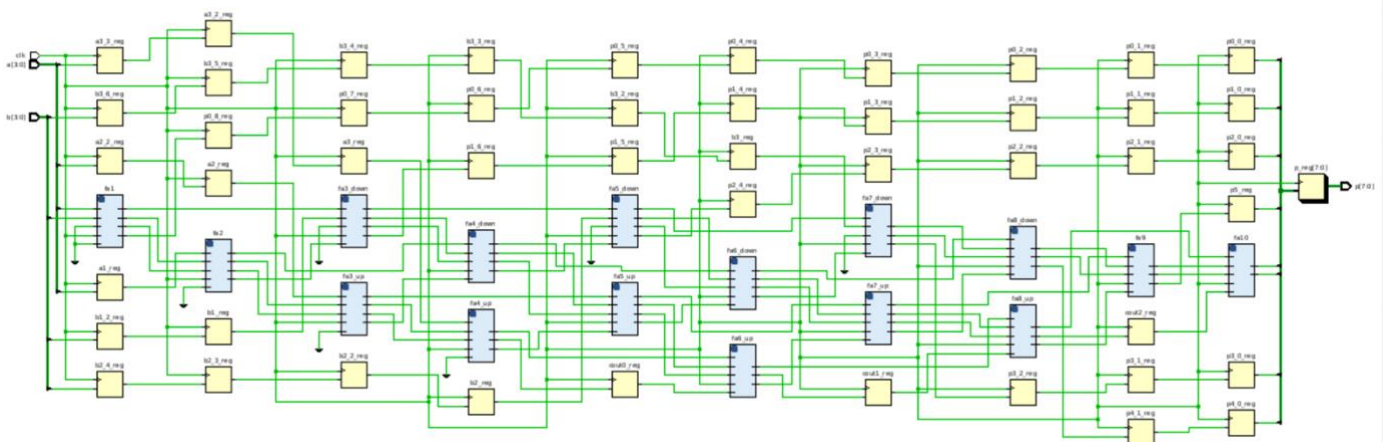
p <= ci3_temp(3) & s3_temp(3) & p5 & p4_0 & p3_0 & p2_0 & p1_0 & p0_0;
end if;
end process;

end mult_arch;

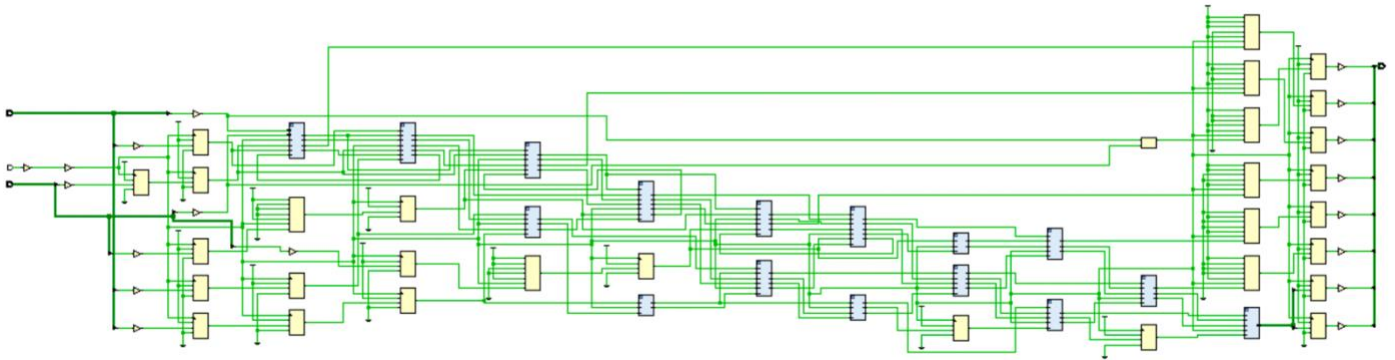
```

Όπου έχουμε ορίσει τους 16 *full_adder_star* και έχουμε υλοποιήσει τις αναγκαίες καθυστερήσεις για τις εισόδους και τις εξόδους του κυκλώματος μας.

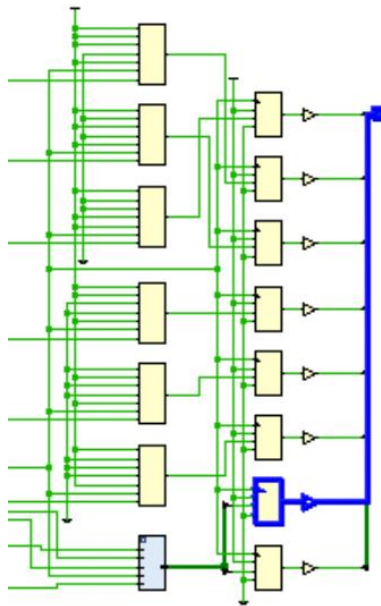
Με τη χρήση του elaborate design λαμβάνουμε:



Και με του synthesis design:



Εδώ μπορούμε να βρούμε και το critical path μας όπου:



Με χρονική καθυστέρηση 4.076ns όπως φαίνεται και παρακάτω:

Name	Slack ^ 1	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	2	2	1	p_reg[6]/C	p[6]	4.076	3.276	0.800
Path 2	∞	2	2	1	p_reg[7]/C	p[7]	4.076	3.276	0.800
Path 3	∞	2	2	1	p_reg[0]/C	p[0]	4.058	3.258	0.800
Path 4	∞	2	2	1	p_reg[1]/C	p[1]	4.058	3.258	0.800
Path 5	∞	2	2	1	p_reg[2]/C	p[2]	4.058	3.258	0.800
Path 6	∞	2	2	1	p_reg[3]/C	p[3]	4.058	3.258	0.800
Path 7	∞	2	2	1	p_reg[4]/C	p[4]	4.058	3.258	0.800

Τέλος, για να δούμε αν λειτουργεί όντως σωστά το κύκλωμα μας, πρέπει να γράψουμε ένα αντίστοιχο testbench:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity mult_tb is
end mult_tb;

architecture mult_tb_arch of mult_tb is

component mult
    port(
        a : in std_logic_vector(3 downto 0);
        b : in std_logic_vector(3 downto 0);
        p : out std_logic_vector(7 downto 0);
        clk : in std_logic
    );
end component;

signal test_a, test_b: std_logic_vector(3 downto 0);
signal test_p : std_logic_vector(7 downto 0);
signal test_clk : std_logic := '0';

constant clk_period : time := 10 ns;

begin

    uut: mult
        port map (
            a => test_a,
            b => test_b,
            p => test_p,
            clk => test_clk
        );

    test: process
    begin
        for i in 0 to 15 loop
            test_a <= std_logic_vector(to_unsigned(i, 4));
            for j in 0 to 15 loop
                test_b <= std_logic_vector(to_unsigned(j, 4));
                wait for clk_period;
            end loop;
        end loop;

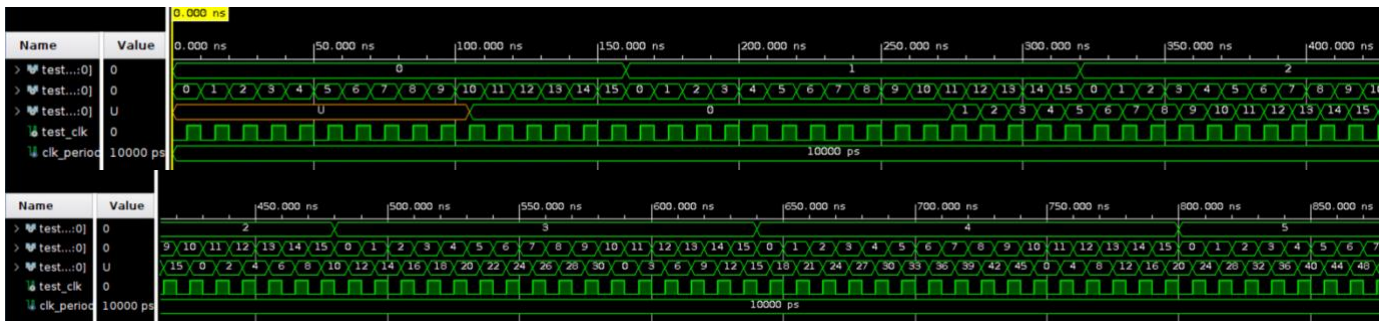
        wait;
    end process;

    Generate_clock: process
    begin
        test_clk <= '0';
        wait for clk_period / 2;
        test_clk <= '1';
        wait for clk_period / 2;
    end process;

end mult_tb_arch;
```

Το οποίο κάνει iterate σε όλες τις πιθανές εισόδους.

Αν τώρα τρέξουμε το simulation θα λάβουμε:



Το οποίο όντως βγάζει το πρώτο αποτέλεσμα 10 κύκλους ρολογιού (δηλαδή το $T_{latency}$ μας) μετά από όταν λάβαμε την πρώτη είσοδο μας, και αναπαριστά όντως τον πολλαπλασιασμό των διανυσμάτων a, b .